

Übung 2

Mustererkennung und Maschinelles Lernen – SS 2026

Frist: 20.05.25

Allgemeine Hinweise

- Bereiten Sie Ihre Lösungen soweit es geht als Jupyter-Notebook (.ipynb) vor (inkl. Plots und Ihren Beobachtungen).
- Sie können fertige Funktionen aus Standardbibliotheken (numpy, matplotlib, pandas, scikit-learn, scipy, statsmodels, ...) nutzen.
- Nutzen Sie das beiliegende Code-Gerüst mit Imports, Random Seeds und Datenerzeugung.

Aufgabe 1: Polynomielle Regression

Ziel ist, das klassische Lehrbeispiel aus der Vorlesung nachzuvollziehen und Overfitting selbst zu sehen. Die Daten sind im Notebook bereits vorbereitet.

(a) Datenexploration

Im Gerüst werden ein Trainingsdatensatz (N=10) und ein Testdatensatz (N=100) erzeugt – beide stammen aus der Funktion $\sin(2\pi x)$ plus normalverteiltem Rauschen mit $\sigma=0,3$.

Plotten Sie beide Datensätze gemeinsam mit der wahren Funktion $\sin(2\pi x)$.

Warum macht es Sinn, dass der Trainingssatz bewusst klein ist?

(b) Polynomielle Regression

Passen Sie für $M \in \{0, 1, 3, 9\}$ jeweils ein Polynom an die Trainingsdaten an. Sie können hierzu `numpy.polyfit` benutzen.

Plotten Sie alle vier Anpassungen, jeweils gemeinsam mit den Trainingspunkten und der wahren Funktion.

(c) Lernkurve

Berechnen Sie für $M = 0, 1, \dots, 9$ den RMS-Fehler auf Trainings- und Testdaten:

$$E_{\text{RMS}} = \sqrt{(1/N) * \sum (y_{\text{pred}} - t)^2)}$$

Plotten Sie beide Kurven (Train/Test) im selben Diagramm gegen M . Bestimmen Sie den „Sweet Spot“ (das M mit dem kleinsten Test-Fehler).

Was passiert bei $M=9$? Wie unterscheiden sich die Verläufe von Train- und Test-Fehler? Warum?

(d) Regularisierung (Ridge)

Bishop ergänzt die Fehlerfunktion um einen Strafterm:

$$\tilde{E}(w) = (1/2) \sum (y(x_n, w) - t_n)^2 + (\lambda/2) ||w||^2$$

Vergleichen Sie den Fehler für $M=9$ mit drei Werten: $\lambda \in \{0, e^{-18}, 1\}$ (also kein, schwacher, starker Strafterm). Sie können hier `sklearn.linear_model.Ridge` als Teil einer scikit-learn [Pipeline](#) verwenden.

Plotten Sie alle drei resultierenden Kurven und stellen Sie die Koeffizientenvektoren als Tabelle dar. Was passiert mit der Größenordnung der Koeffizienten, wenn λ wächst?

(e) Mehr Daten vs. Regularisierung

In der Vorlesung wurde folgende Heuristik genannt: „Die Anzahl der Trainingsdatenpunkte sollte mindestens ein Vielfaches (z. B. 5 oder 10) der Anzahl der Parameter des Modells betragen.“ Wiederholen Sie die Anpassung mit $M=9$, aber diesmal $N=100$ Trainingspunkten (ohne Regularisierung). Vergleichen Sie das Resultat mit der regularisierten Lösung aus (d) bei $N=10$.

Welche der beiden Strategien – „mehr Daten“ oder „Regularisierung“ – würden Sie in der Praxis bevorzugen, und wovon hängt die Antwort ab?

Aufgabe 2: Entscheidungsbaum und Random Forest

Wir verwenden den Wine-Datensatz, der direkt in scikit-learn verfügbar ist. Die Aufgabe ist, das Weinanbaugebiet (3 Klassen) aus 13 chemischen Inhaltsstoffen vorherzusagen.

```
from sklearn.datasets import load_wine
data = load_wine()
X, y = data.data, data.target
```

(a) Datenexploration

- Untersuchen Sie Struktur von X , die Klassennamen, die Featurenamen und die Klassenverteilung.
- Erstellen Sie eine kleine Tabelle (DataFrame), die für 3 selbst gewählte Features die Mittelwerte je Klasse zeigt. Erkennen Sie bereits visuell Features, die die Daten potentiell gut trennen?

(b) Train/Test-Split und Baseline-Baum

Splitten Sie 70/30 mit `sklearn.model_selection.train_test_split(..., stratify=y, random_state=42)`. Trainieren Sie einen `DecisionTreeClassifier()` mit Default-Parametern. Berichten Sie: Test-Accuracy, Confusion-Matrix (`sklearn.metrics.confusion_matrix`), Tiefe des resultierenden Baums (`clf.get_depth()`).

(c) Overfitting in Bäumen

Variieren Sie `max_depth` von 1 bis 15. Plotten Sie Train- und Test-Accuracy gegen die Tiefe in einem Diagramm. Bei welcher Tiefe beginnt Overfitting sichtbar zu werden?

(d) Visualisierung des Baums

Trainieren Sie einen Baum mit `max_depth=3` und visualisieren Sie ihn mit `sklearn.tree.plot_tree(clf, feature_names=..., class_names=..., filled=True)`. Beschreiben Sie in 2–3 Sätzen, welche Entscheidung an der Wurzel getroffen wird und warum dieser Split sinnvoll ist (Stichwort: Information Gain).

(e) Random Forest

Trainieren Sie einen `RandomForestClassifier(n_estimators=100, oob_score=True, random_state=42)`. Vergleichen Sie:

- Test-Accuracy des Random Forest vs. Test-Accuracy des Einzelbaums aus (b)
- OOB-Score (`clf.oob_score_`) – was misst dieser konkret?

Aufgabe 3: K-Means — von Hand und in Code

(a) K-Means händisch

Gegeben sei der folgende **eindimensionale** Datensatz:

```
X = { 1, 2, 3, 8, 9, 10, 25 }
```

Führen Sie **zwei vollständige Iterationen** des K-Means-Algorithmus mit $K=2$ und Initialisierung $\mu_1^{(0)} = 2$, $\mu_2^{(0)} = 9$ **per Hand** durch (auf Papier oder in Markdown — kein Code). Stellen Sie tabellarisch für jede Iteration dar:

- die Zuordnungen r_{nk} (zu welchem Cluster gehört jeder Punkt?)
- die neuen Centroide μ_1, μ_2
- den Wert der Zielfunktion $J = \sum_n \sum_k r_{nk} \cdot \|x_n - \mu_k\|^2$

Zeigen Sie, dass J monoton fällt. Beobachten Sie zusätzlich: Was passiert mit dem zweiten Centroid wegen des Punktes $x=25$? Welcher Nachteil von K-Means wird hier sichtbar?

(b) K-Means auf synthetischen Daten

Im Starter-Notebook ist ein 2D-Datensatz mit `make_blobs` erzeugt (4 Cluster, 300 Punkte). Wenden Sie `sklearn.cluster.KMeans(n_clusters=4, n_init=10, random_state=42)` darauf an. Plotten Sie:

- Datenpunkte gefärbt nach finaler Cluster-Zuordnung
- die finalen Cluster-Zentren als große Marker (z. B. schwarze Sterne)

Geben Sie zusätzlich die finale Trägheit (`kmeans.inertia_`) aus – das ist genau der J -Wert aus (a).

(c) Elbow-Methode

Trainieren Sie K-Means für $K = 1, 2, \dots, 10$ und plotten Sie die Zielfunktion J gegen K (Elbow-Plot). Welches K würden Sie aufgrund des Plots wählen? Stimmt es mit der Wahrheit ($K=4$) überein?

(d) Initialisierungs-Sensitivität

Wiederholen Sie K-Means mit $K=4$ **fünfmal** mit $n_init=1$, $init='random'$ und unterschiedlichen $random_state$ -Werten (z. B. 0 bis 4). Berichten Sie die fünf finalen Trägheits-Werte als Tabelle. Vergleichen Sie zusätzlich mit dem Default-Lauf ($n_init=10$, $init='k-means++'$). Sind die Werte alle gleich? Was bedeutet das in Verbindung mit der Aussage, K-Means „findet das Optimum“?

Hinweis: Der Default $init='k-means++'$ ist eine intelligente Initialisierungsstrategie, die schon bei einem einzelnen Lauf erstaunlich oft das Optimum findet.

Aufgabe 4: Vorteile von GMMs

(a) Datensatz mit elliptischen Clustern

Im Starter-Notebook ist ein Datensatz aus zwei 2D-Gauß-Verteilungen mit unterschiedlichen, rotierten Kovarianzstrukturen erzeugt. Plotten Sie den Datensatz (gefärbt nach wahrer Klasse) und beschreiben Sie kurz, was Sie sehen.

(b) K-Means vs. GMM

Wenden Sie auf denselben Datensatz beide Algorithmen an:

```
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit(X)
gmm = GaussianMixture(n_components=2, covariance_type='full', random_state=42).fit(X)
```

Plotten Sie beide Ergebnisse, jeweils mit den geschätzten Cluster-Zentren bzw. -zuordnungen.

(c) Interpretation des Ergebnis

Warum versagt K-Means bei elliptischen Clustern?